

- `track_functions` gathers statistics about the use of functions and stored procedures.
- `track_io_timing` allows us to count the time spent in different input/output operations.
- `stat_temp_directory` is the (relative) directory name to use as temporary storage for statistics collection.

Modifying the configuration from a live system

It is possible to modify the cluster configuration from within a database connection by means of the `ALTER SYSTEM` command.

`ALTER SYSTEM` provides us with a SQL way to set a parameter value, and the parameter will be appended to the special file, `postgresql.auto.conf`, which lives within the `PGDATA` directory. The `postgresql.auto.conf` file is loaded automatically, at the end, when the server boots or a **reload** signal (HUP) is issued. Therefore, parameters contained in `postgresql.auto.conf` will take priority over those in `postgresql.conf` and the end result will be that the changes will be applied as if you have manually edited the `postgresql.conf` file.

`ALTER SYSTEM` can only be executed from a database administrator. For example, let's say you issue the following command:

```
forumdb=# ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM
```

The end result will be to have a `postgresql.auto.conf` file that looks as follows:

```
$ cat /postgres/16/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
archive_mode = 'on'
```

As you can see, the changed parameter was placed in the file as you manually edited it. The file contains a warning banner about the fact that you should not edit it manually because the system will not take your changes into account and will overwrite its content.

It is also possible to specify `DEFAULT` as the value for an option, so that option will be removed from the `postgresql.conf.auto` file. The `ALTER SYSTEM` also supports `RESET` to reset a setting to its previous value, or `RESET ALL` to reset all the settings from `postgresql.auto.conf`.

Assuming we only changed the `archive_mode` as in the previous example, the following two commands are equivalent and result in removing the changed settings from the `postgresql.auto.conf` file:

```
forumb=# ALTER SYSTEM SET archive_mode TO DEFAULT;
ALTER SYSTEM
forumb=# ALTER SYSTEM RESET archive_mode;
ALTER SYSTEM
```

The following input will remove *every* changed setting in `postgresql.auto.conf`:

```
forumb=# ALTER SYSTEM RESET ALL;
ALTER SYSTEM
```

Configuration generators

Instead of starting from the annotated `postgresql.conf` file and tuning it by yourself, you can exploit an automated tuning system to get a configuration from scratch. Such configurations could be good enough, or at least a starting point for more improvements, depending on your needs and how good the tool that produced it is.

A good configuration system is **PGConfig**, an online system available at <https://www.pgconfig.org> where you can specify the main settings of the host serving your cluster, such as memory, hard disk type, concurrency, and so on.

With those few details, as shown in the following screenshot, the system can produce different configurations, depending on the workload you are going to use the cluster for:



PostgreSQL
Configuration Builder

[Home](#) [Contribute](#) [Documentation](#)

Server

Operating system
GNU/Linux Based

Architecture
64 Bits (x86-64)

Storage type
SSD Storage

Number of CPUs
2

Total Memory (GB)
4

Max connections
100

Database

Application profile
General web applications

PostgreSQL Version
15 (Latest)

[Profile Comparison](#) [Export Config](#)

Memory Configuration

	Default Value	WEB	OLTP	DW	MIXED	DESKTOP
> shared_buffers	128MB	1GB	1GB	1GB	512MB	256MB
> effective_cache_size	4GB	3GB	3GB	3GB	2GB	614MB
> work_mem	4MB	10MB	14MB	20MB	4MB	839KB
> maintenance_work_mem	64MB	205MB	205MB	205MB	102MB	41MB

Checkpoint Related Configuration

	Default Value	WEB	OLTP	DW	MIXED	DESKTOP
> min_wal_size	80MB	2GB	2GB	2GB	2GB	2GB
> max_wal_size	1GB	3GB	3GB	3GB	3GB	3GB

Figure 16.1: The PGConfig PostgreSQL automatic configuration generator

The tool allows you to select multiple pre-packaged configurations, for example, **On-Line Transactional Processing (OLTP)** or **On-Line Analytical Processing (OLAP)**, or even a web application.

Once you have selected the configuration that best fits your workload, you can export such a configuration as a `postgresql.conf` file or as a set of `ALTER SYSTEM` statements to be executed as a SQL interactive script so that you can apply the configuration to your cluster:

The screenshot shows the 'Export Config' tab in the PGConfig application. At the top, there are two tabs: 'Profile Comparison' and 'Export Config'. Below the tabs, there are two dropdown menus: 'Export Format' set to 'UNIX-like config file' and 'Log Format' set to 'Comma-separated values'. To the right of these is a toggle switch for 'Include PGBadger log configuration' which is currently turned on, and a 'Copy to clipboard' button. The main area displays a generated configuration file with the following content:

```
# Generated by PGConfig 3.1.2 (5c1bbd5df8b2e1a63ef02bd759ef59016a923cc)
# https://api.pgconfig.org/v1/tuning/get-config?format=conf&include_pgbadger=true&Log_format=csvLog&max_connections=100&pg_version=16&environment_name=WEB&total_ram=4GB

# Memory Configuration
shared_buffers = 1GB
effective_cache_size = 3GB
work_mem = 10MB
maintenance_work_mem = 205MB

# Checkpoint Related Configuration
min_wal_size = 2GB
max_wal_size = 3GB
checkpoint_completion_target = 0.9
wal_buffers = -1

# Network Related Configuration
listen_addresses = '*'
max_connections = 100

# Storage Configuration
random_page_cost = 1.1
effective_io_concurrency = 200

# Worker Processes Configuration
max_worker_processes = 8
max_parallel_workers_per_gather = 2
max_parallel_workers = 2

# Logging configuration for pgbadger
logging_collector = on
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
lc_messages = 'C'

# Adjust the minimum time to collect the data
log_min_duration_statement = '10s'
log_autovacuum_min_duration = 0

# CSV Configuration
log_destination = 'csvlog'
```

Figure 16.2: An example of an automatically generated configuration

As you can see, the result is a bunch of configuration parameters that you can copy and paste into a “blank” configuration file. The idea is to start from this configuration and continue tuning on top of it.

The following screenshot shows the very same configuration but produced by means of ALTER SYSTEM statements; that is, you can apply the configuration as a SQL script, depending on your needs:

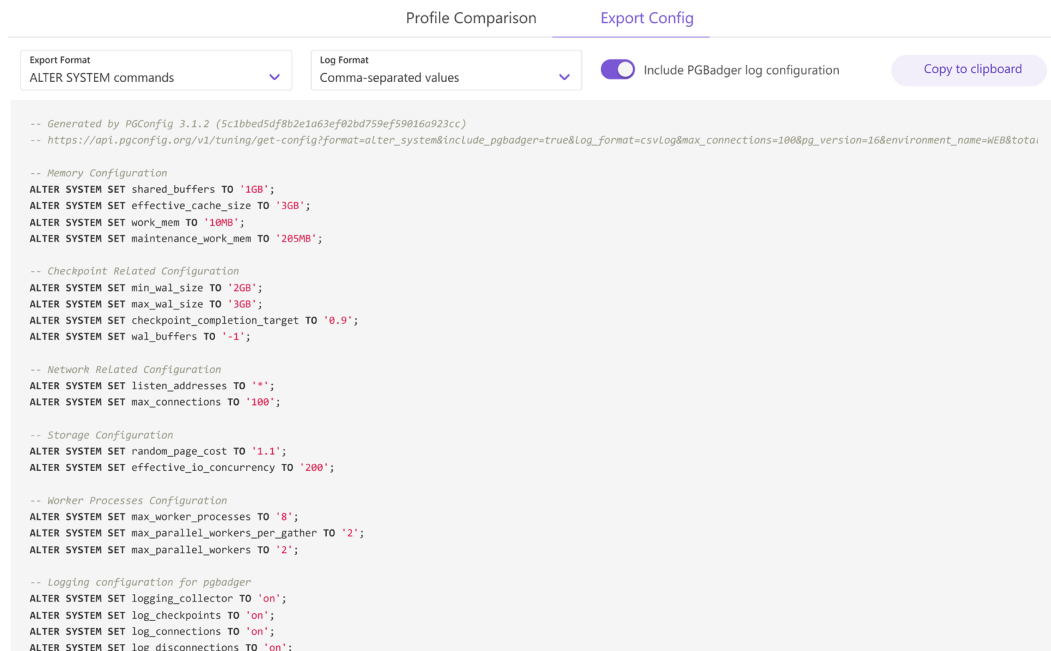


Figure 16.3: An example of an automatically generated configuration that is based on ALTER SYSTEM commands

PGConfig is just one option you can use to get a customized configuration that you can start working on. Of course, there is no need to use it since PostgreSQL comes with a default configuration, and this configuration generator does not represent a “silver bullet” to provide you with the optimal configuration for your cluster. In any case, you will need to tune and fix your parameters to optimize the cluster, depending on your needs, workload, and hardware.

In the next section, you are going to discover how to monitor your cluster, as well as how to discover bottlenecks and problems that can be fixed by tuning your queries or cluster configuration.

Monitoring the cluster

Monitoring the cluster allows you to understand what the cluster is doing at any given point in time and potentially act and react accordingly to avoid degradation in the performance and usability of databases. PostgreSQL provides a rich set of catalogs that allow a database administrator to monitor the overall activity by issuing only SQL statements and queries.

You can also combine the results of the information coming from the catalog with other external monitoring tools, ranging from your operating system's tools to more complex ones such as Nagios.

In this section, we will have a look at the main PostgreSQL catalogs used to monitor and collect information about database activities. As you can imagine, only a database administrator can get complete information about overall cluster activities.

The cluster collects information about activities by means of the **statistic collector**, a dedicated process that is responsible for collecting (and therefore, providing) information in a cluster-wide way. Statistics are not in real time, even if you feel they are. This is because statistics are updated no more frequently than every 500 milliseconds by backend processes, assuming they are idle. Moreover, statistics within a transaction block are “frozen,” meaning you cannot observe changes in the statistics unless your transaction has finished.

Statistics are kept across clean shutdowns and restarts of the cluster, but in the case of recovering from a crash, all the statistics are deleted and collection starts from scratch. There is also the possibility to manually reset the statistics for a specific database by invoking the `pg_stat_reset()` function as a database superuser.

In the following subsections, we will concentrate on a set of statistics that can be helpful in understanding what is going on within the cluster to help monitor the overall activity.

Information about running queries and sessions

The `pg_stat_activity` catalog provides one tuple for every backend process active in the cluster and, therefore, for every client connected. The following simple queries provide a detailed output:

```
forumdb=# SELECT username, datname, client_addr, application_name,
               backend_start, query_start,
               state, backend_xid, query
FROM pg_stat_activity;
...
```

[RECORD 4]-----	
username	luca
datname	forumdb
client_addr	192.168.222.1
application_name	psql

```

backend_start      | 2023-09-13 16:42:50.9931+02
query_start        | 2023-09-13 16:44:20.601118+02
state              | idle
backend_xid        |
query              | INSERT INTO tags( tag ) SELECT 'A Fake Tag' FROM
generate_series( 1, 10000 );

```

As you can see, the user `luca` (in the `username` field) is connected via `psql` (the `application_name` field) from a remote host (the `client_addr` field) and executed the `INSERT INTO` query called `tags` over the `forumdb` database. It is interesting to note the `state` field, which reports the status of the running query. In the preceding example, it says `idle`, meaning that the query is waiting for something else to happen, and may even be finished.

It is important to note that `pg_stat_activity` only reports the very last executed query from a session or connection. Remember that the catalog shows a tuple for every connected client and that the statistics are not updated until a new statement is executed.

The `pg_stat_activity` catalog can be queried by anyone, even normal users, but the amount of information reported could be trimmed out depending on the privileges of the user executing the query.

Inspecting locks

The `pg_locks` special catalog provides a clear and detailed view of any locks that are acquired by different transactions and statements. The idea is that by inspecting this catalog, the system administrator can get a glance at possible bottlenecks and competition among transactions. It is useful to query this catalog by joining it with `pg_stat_activity` in order to get more detailed information about what is going on. The following is an example of a query and a partial result:

```

forumdb=# SELECT a.username, a.application_name, a.datname, a.query,
              l.granted, l.mode
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid;
...
-[ RECORD 5 ]-----+-----
--
username          | luca
application_name   | psql
datname            | forumdb

```

```

query          | delete from tags;
granted         | t
mode           | RowExclusiveLock
...
-[ RECORD 9 ]----+-----
--
username       | luca
application_name | psql
datname        | forumdb
query          | insert into tags( tag ) values( 'FreeBSD' );
granted        | t
mode           | ExclusiveLock

```

There are two connections for the user `luca` to the `forumdb` database, and one connection has acquired a lock to delete tuples while the other is inserting tuples into the `tags` table. The `granted` column expresses whether the lock is acquired, so selecting only the non-granted locks is a good starting point to get advice on blocked queries. The `mode` column indicates what kind of lock the query is trying to acquire.

With these suggestions, and thanks again to an accurate join with `pg_stat_activity`, you can find blocked queries, as shown in the following example (this is a continuation of the same scenario depicted previously):

```

forumdb=# SELECT query, backend_start, xact_start, query_start,
              state_change, state,
              now()::time - state_change::time AS locked_since,
              pid, wait_event_type, wait_event
FROM pg_stat_activity
WHERE wait_event_type IS NOT NULL
ORDER BY locked_since DESC;
...
-[ RECORD 6 ]---+-----
query          | insert into tags( tag ) values( 'FreeBSD' );
backend_start   | 2023-09-14 08:26:57.762887+02
xact_start      | 2023-09-14 08:27:00.017983+02
query_start     | 2023-09-14 08:27:14.745784+02
state_change    | 2023-09-14 08:27:14.775535+02
state           | idle in transaction

```

```

locked_since | 00:07:33.411832
pid          | 60239
wait_event_type | Client
wait_event    | ClientRead

```

As you can see, the query has been waiting for 7 minutes and 33 seconds (the `locked_since` column), but the query is idle in transaction (the `state` column) and is waiting for input from a client (the `wait_event` and `wait_event_type` columns). In other words, the query is waiting for the user to complete (either `COMMIT` or `ROLLBACK`) the transaction.

Taking advantage of `pg_locks` can help you follow the evolution of transactions and their contention, as well as decide on how to terminate queries that are blocking other workloads.

There is also a commodity function named `pg_blocking_pids()` that accepts a process identifier for a backend and returns a list of process identifiers that are blocking such a process.

Inspecting databases

You can get detailed information about the status of your databases by querying the `pg_stat_database` special catalog. This catalog provides information about `COMMIT` and `ROLLBACK` transactions, deadlocks, and conflicts. Please consider that deadlocks and rollbacks are natural events in a database, but if you see the numbers grow quickly, this could mean there's been an application error or that there are clients who are trying to do things incorrectly in a database and thus are forced to roll back.

As an example, by using the following query, you can get details about your databases:

```

forumdb=# SELECT datname, xact_commit, xact_rollback, blks_read,
conflicts, deadlocks,
           tup_fetched, tup_inserted, tup_updated, tup_deleted, stats_reset
FROM pg_stat_database;

...
-[ RECORD 6 ]+-----
datname      | forumdb
xact_commit  | 802
xact_rollback | 9
blks_read    | 1800
conflicts    | 0
deadlocks    | 0
tup_fetched  | 32977

```

```
tup_inserted | 1391
tup_updated  | 46
tup_deleted  | 0
stats_reset  |
```

As you can see, the `forumdb` database doesn't have any conflicts or deadlocks, and the number of committed transactions (the `xact_commit` column) is much higher than the number of aborted transactions (the `xact_rollback` column). Therefore, we can assume that the database is fine and that the applications are issuing good queries.

The last column, `stats_reset`, is particularly important since it indicates whenever the statistics information for a database has been reset, meaning deleted. Knowing how much time has elapsed since the statistics have been reset helps in validating the database. If the column is empty, the database statistics have never been reset manually.

Inspecting tables and indexes

The `pg_stat_user_tables` and `pg_stat_user_indexes` special catalogs provide detailed information about the usage of a table or an index, such as the number of tuples, the number of reads and writes, and so on.

In order to better understand, consider a table where some transactions have been deleted or updated, and new records inserted; the following query provides detailed information about the status of that table:

```
forumdb=# SELECT relname, seq_scan, idx_scan,
               n_tup_ins, n_tup_del, n_tup_upd, n_tup_hot_upd,
               n_live_tup, n_dead_tup,
               last_vacuum, last_autovacuum,
               last_analyze, last_autoanalyze
FROM pg_stat_user_tables;
```

```
-[ RECORD 1 ]-----+-----
relname      | tags
seq_scan     | 20
idx_scan     | 0
n_tup_ins    | 100007
n_tup_del    | 63
```